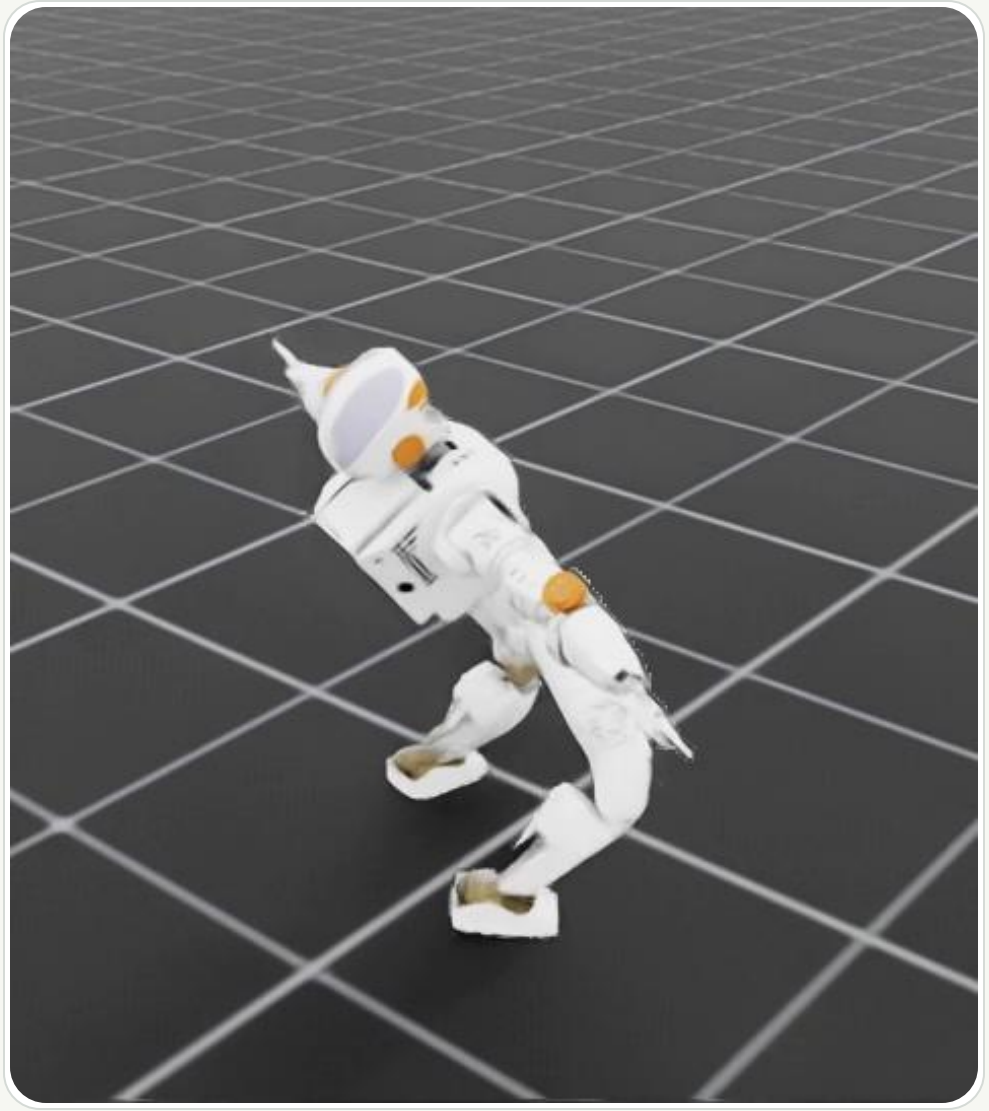
FINAL PROJECT SUBMISSION

EasyRTG

A no-code training ground for robot learning



Shivam Chavan · 3053682

Supervisor: Shraddha Naik

Project Motivation

- **In my final year of engineering**

  I set myself a harder project than I needed to: build a physical robot while alsop training the simulated version of it with reinforcement learning.

- **It took three months**

  Three months of work before I had something I could show anyone. Almost all of that time went into the robot behavior and into wiring the tools together.

- **The project worked**

  I finished it, and the robot learned what I wanted. But I could see what the three months had really been spent on.

- **That is the gap**

  Anyone new to this cannot get started at all unless they already understand the algorithms and can write the code. The knowledge needed to begin is the barrier, here

So, I built the application I would've wanted when I started

Literature Survey

Area	What already exists	Project application
Algorithms	PPO (Schulman et al., 2017), SAC (Haarnoja et al., ICML 2018), TD3 (Fujimoto et al., ICML 2018)	Reliable published methods. All three are in this project.
GPU simulation	Isaac Gym (Makoviychuk et al., NeurIPS Datasets 2021), Isaac Lab / Orbit (Mittal et al., RA-L 2023)	Thousands of robots on one GPU. This is the engine we train on.
Legged locomotion	Rudin, Hoeller, Reist & Hutter (CoRL 2022)	Showed a quadruped can be trained in minutes with massively parallel simulation.
Differentiable physics	Brax (Freeman et al., 2021)	Fast and elegant, but our own attempt with it did not produce a usable robot.
Libraries	Stable-Baselines3 (Raffin et al., JMLR 2021), PyBullet (Coumans & Bai)	Well-tested implementations, all written for people who already code.

The gap is that using them still requires an expert understanding.

EasyRTG turns a robot file and a goal into a trained robot

Load → **Describe** → **Configure** → **Train** → **Evaluate**

Open any robot file

Say what it should do

The assistant suggests a setup

Train thousands of copies at once

Watch it and compare results

You can check and approve every important change (human in the loop)

The Underlaying Tech Stack & Features

EasyRTG desktop

Load, set up, watch, compare

Agent + FastAPI

Reads your goal, checks changes, starts runs

NVIDIA Isaac Sim + PPO

Thousands of robots training on the GPU

Saved records

Settings, numbers, saved robots and videos

The assistant helps at every step

Inspect

Reads the robot's joints and limits

→

Translate

Turns your goal into a reward

→

Run & watch

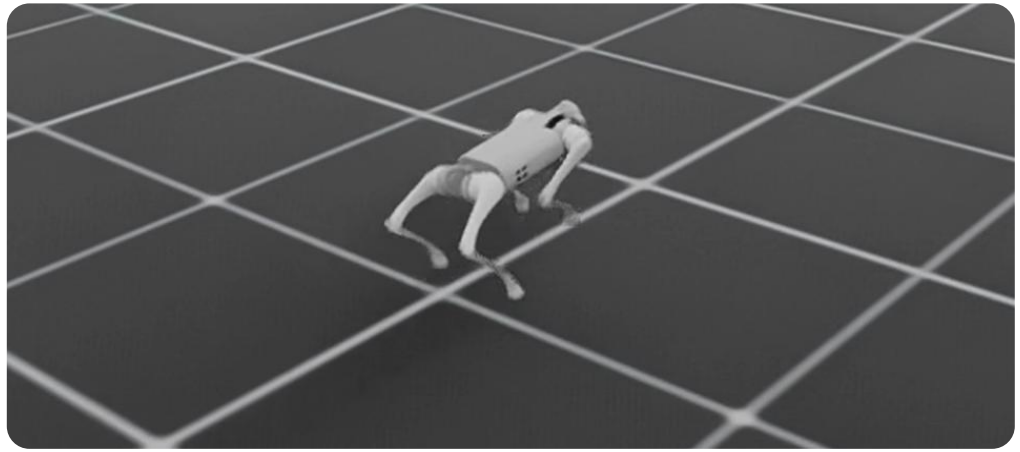
Starts training and reports progress

→

Evaluate (planned)

Runs a saved robot and compares results

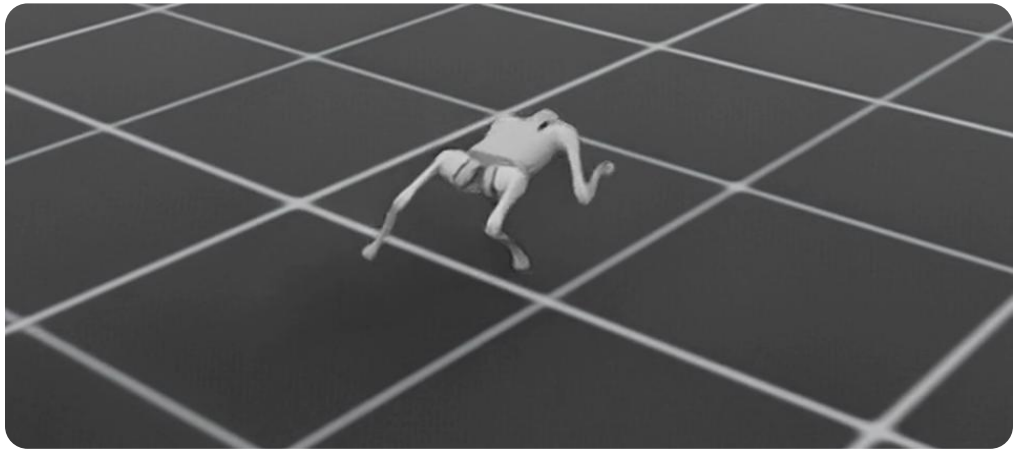
Unitree Go1 robot with two different rewards moves in two different ways



WALKING REWARD · 11 RULES

0.98	963	0.99
metres/second	steps per attempt	how upright

This reward punishes bouncing, so the robot keeps its feet on the ground.



GALLOP REWARD · 7 RULES

1.38	1,000	43%
metres/second	steps per attempt	of time in the air

This reward pays the robot for leaving the ground, and drops the rules against it.

The same Unitree Go1 robot and the same 40 million training steps. Only the reward changed.

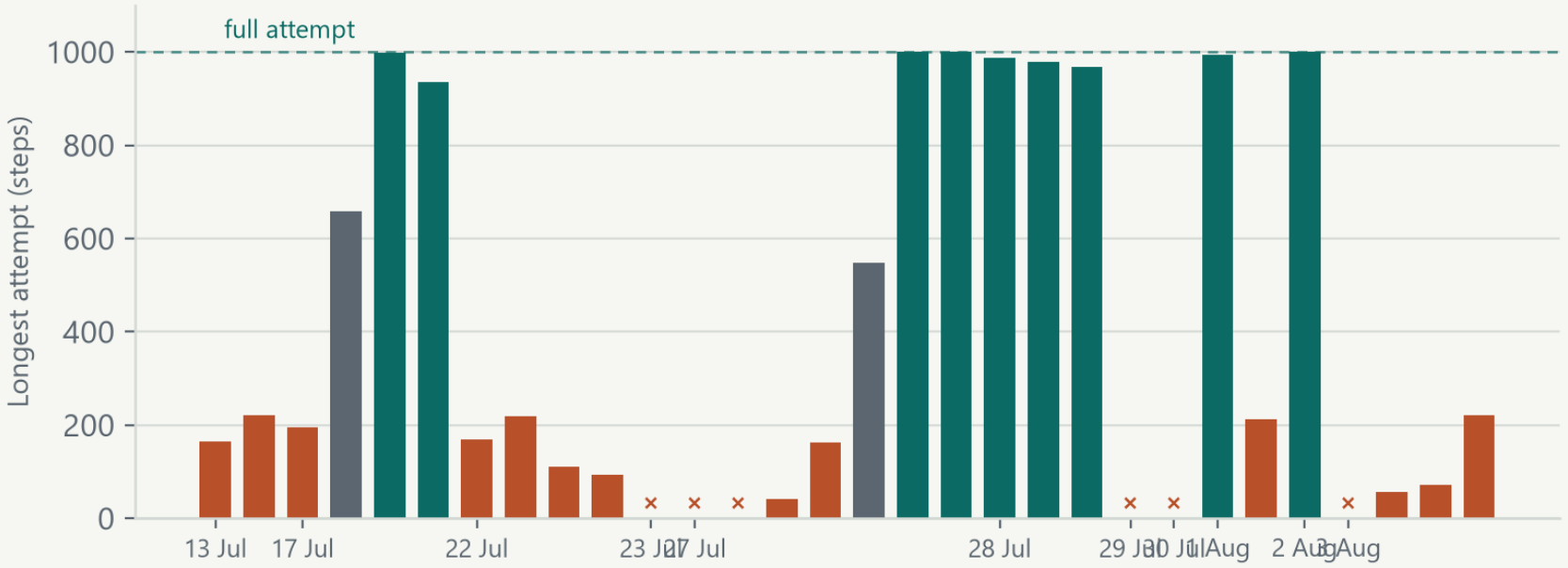
From a fixed setup to one the agent writes

Area	Initial stages	Now
Writing the reward	Users picked from a list of rewards we had written.	• The agent writes the reward as one short line of maths • Any behaviour is possible, not only the ones we shipped
Reading the robot	We guessed the moving parts from words in their names.	• We read the robot's own shape: body, limbs, and which limbs touch the ground • Legs, wheels, arms and humanoids all work.
Standing pose and height	One pose and one height for every robot, copied from a sample.	• We measure each robot's standing pose and height before training • Falling is judged against its own height, not a fixed number
What the reward can see	Only speed, height and how upright the robot was.	• It now feels ground contact for each limb, and time spent in the air • The same reward line works on two legs, four legs or six
Checking a setup	A bad setup only showed up after millions of steps.	• Every setup is checked the moment it is saved • A broken reward fails in seconds instead of after a whole run

Each change moved work out of our code, so the app fits more robots and more tasks without us writing more.

Most of our training runs did not work

Longest attempt each run managed, in date order



30
training runs

9
gave a usable robot

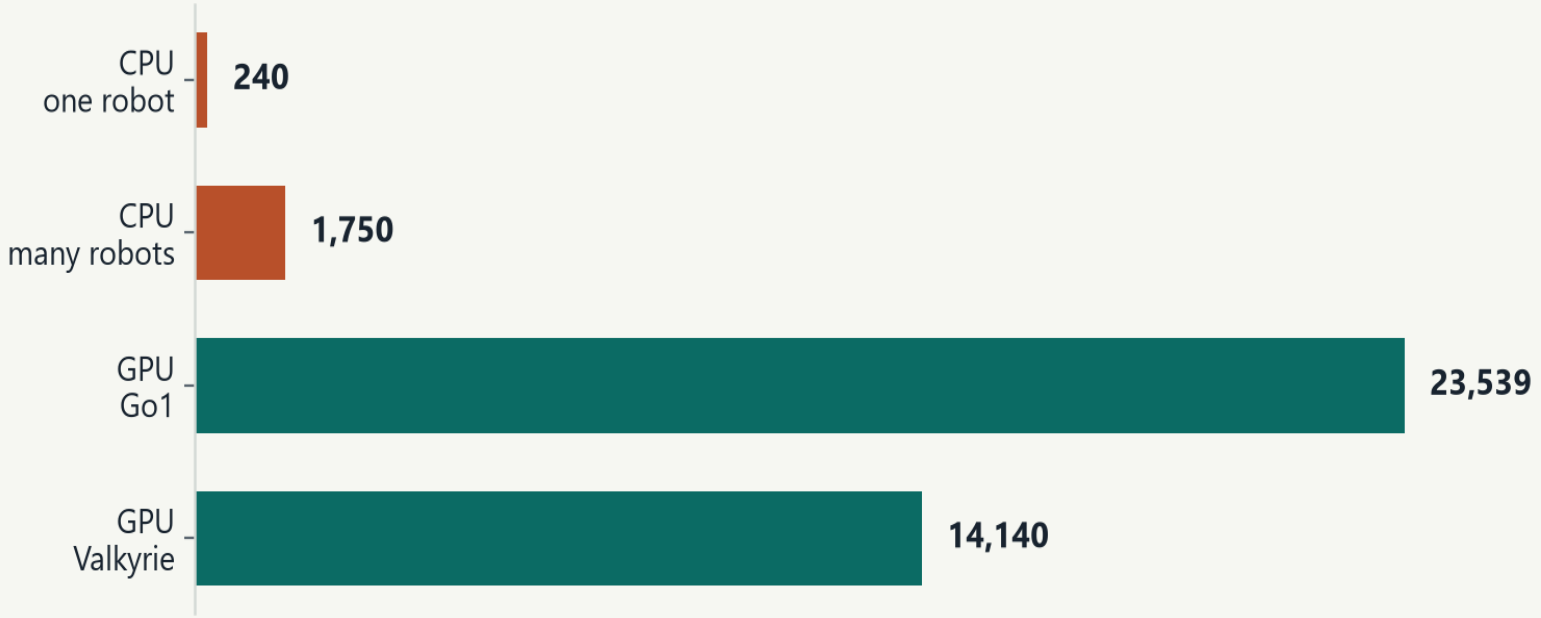
6
crashed at initial stage

Each column is one run. Green finished full 1,000-step attempts, grey learned a little, red fell over quickly. Six columns sit at zero: those runs crashed before training started.

Between 13 July and 3 August. The failures depict how the helping agent is not always accurate.

The GPU changed how fast we could train

Steps of simulation per second



30,138

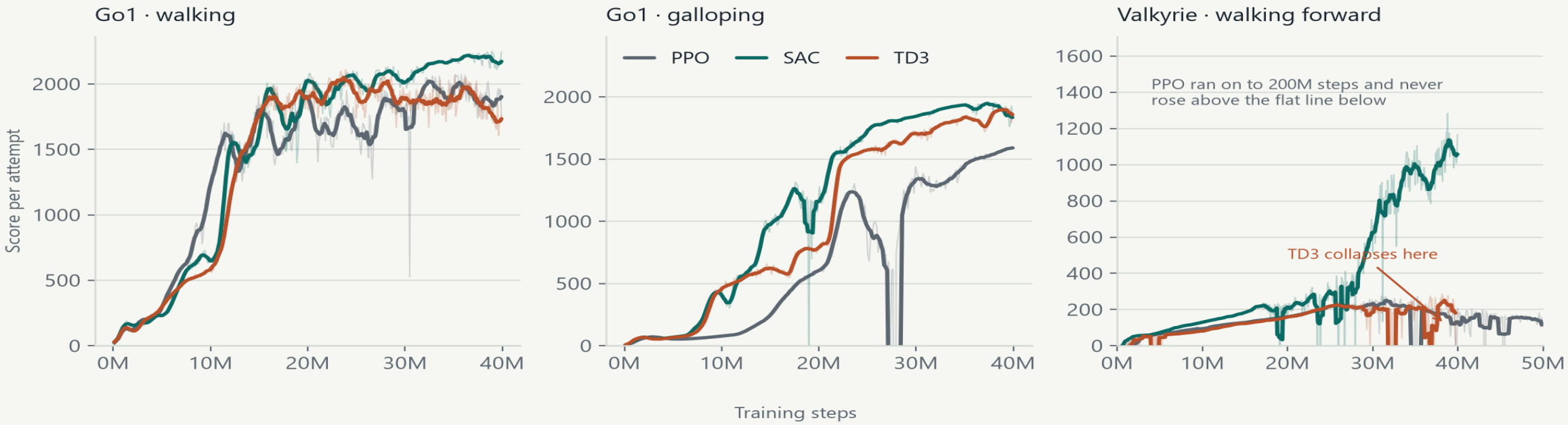
steps a second
in our best run

22.7 min

to train the Go1
for 40 million steps

A longer bar is better. The GPU simulates about a hundred times more steps per second than the CPU did, and better setups took us from 18,000 to 30,138 steps a second across the GPU runs.

All three algorithms on all three tasks

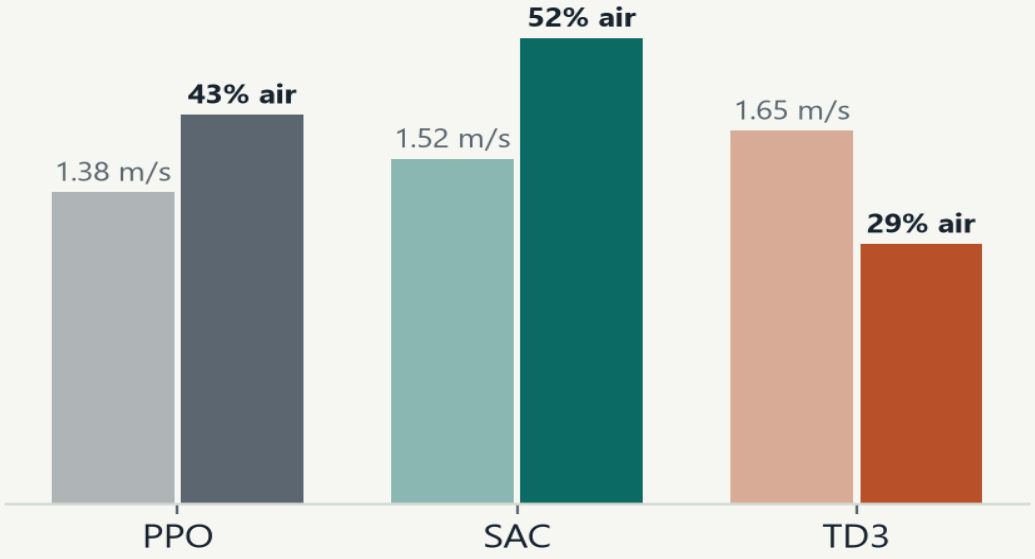


Every task has its own reward, so the three heights are not comparable across the panels.

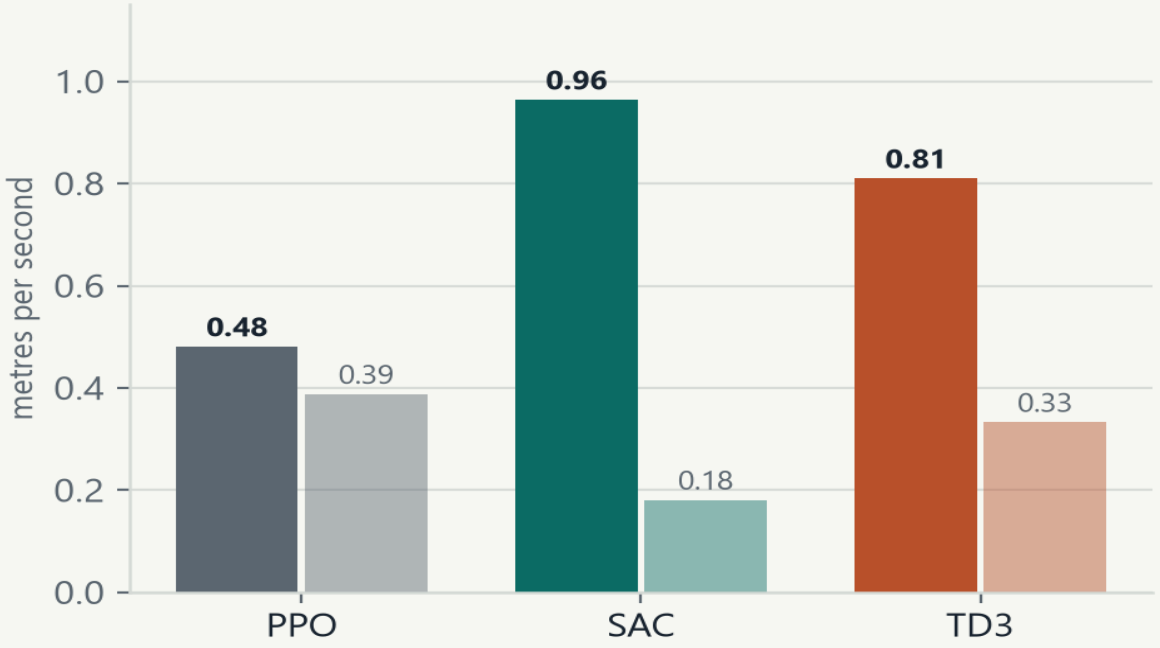
On the two Go1 robot tasks all three algorithms learn. But on the humanoid robot only SAC does.

A similar score does not mean the robot did the same thing

Galloping: speed (pale) against time in the air (solid)



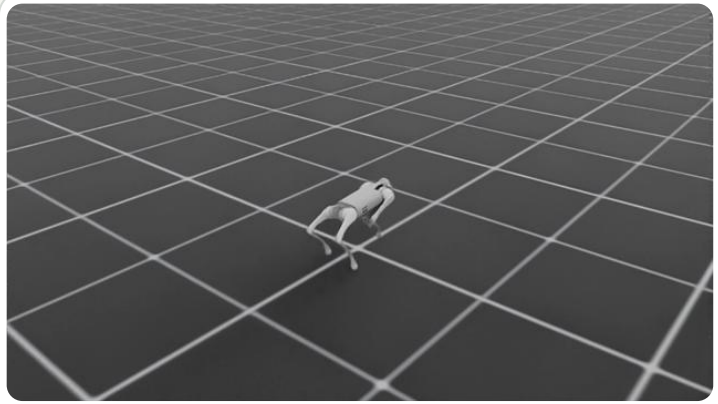
Humanoid: forward (solid) against sideways drift (pale)



On the galloping task TD3 scored within 3% of SAC, and it moves faster, yet it is airborne for 29% of the stride against SAC's 52%: it found a quick trot rather than a gallop.
On the humanoid PPO drifts sideways almost as fast as it walks forward, so its score comes partly from movement that is not the task at all.

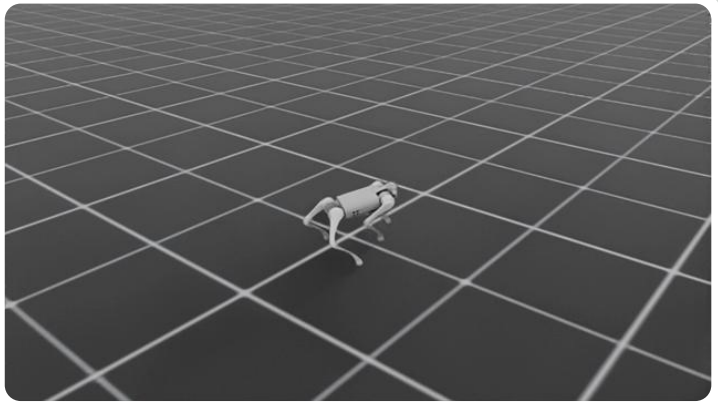
Unitree Go1, walking reward

The task is to walk forward and keep its feet on the ground.



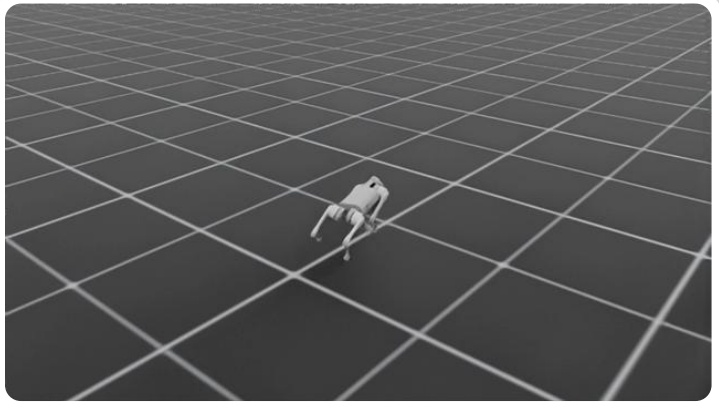
PPO

Walks 818 steps per attempt. Steady, but stops short of the full attempt.



SAC

Walks 995 steps of the 1,000. Most Reward.

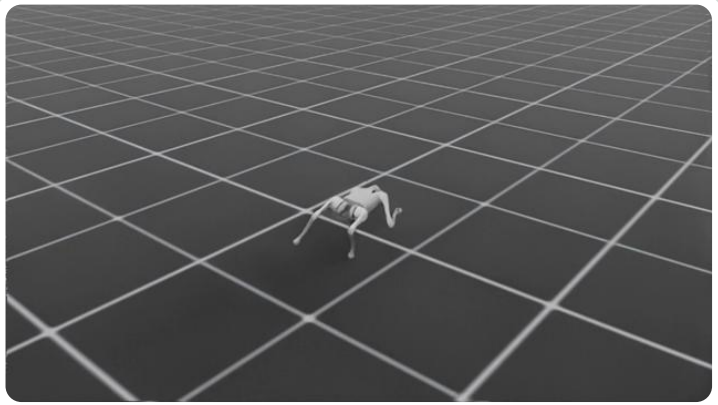


TD3

Walks 809 steps per attempt, level with PPO.

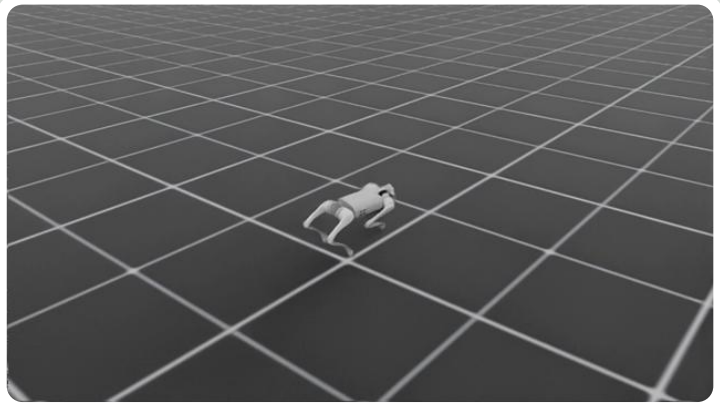
Unitree Go1, galloping reward

The task is to leave the ground, not simply to move fast.



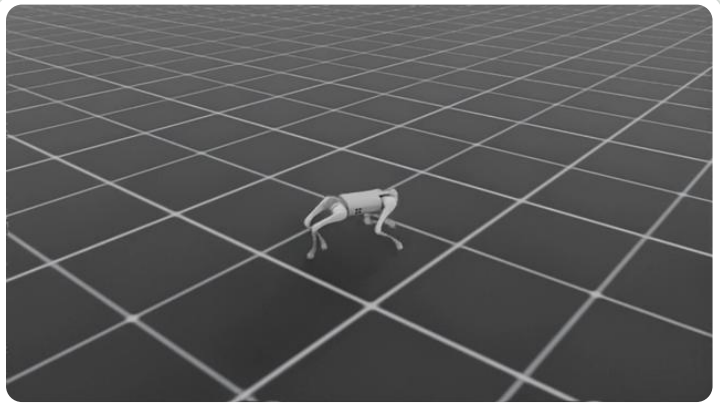
PPO

1.38 m/s with 43% of the stride airborne. A real gallop.



SAC

1.52 m/s and 52% airborne: the clearest gallop we trained.

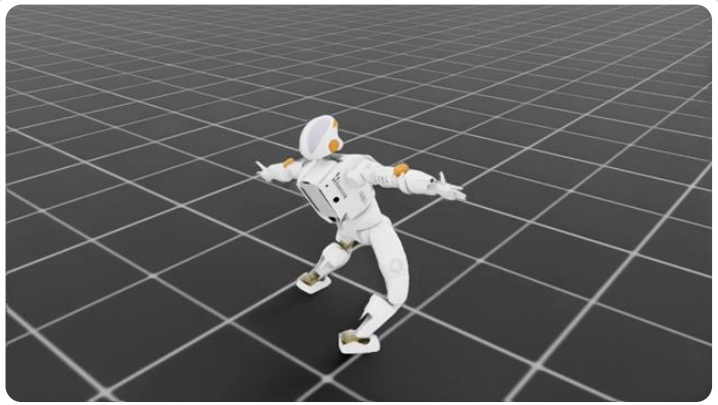


TD3

Fastest at 1.65 m/s, but only 29% airborne. This is a quick trot.

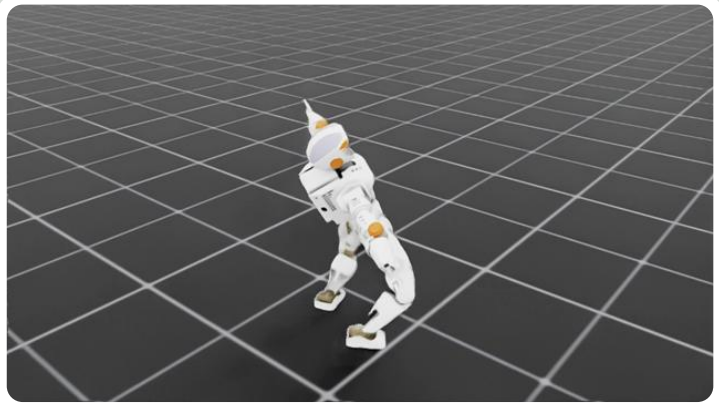
Valkyrie humanoid, walking forward

The hardest of the three tasks, and the one that separates the algorithms.



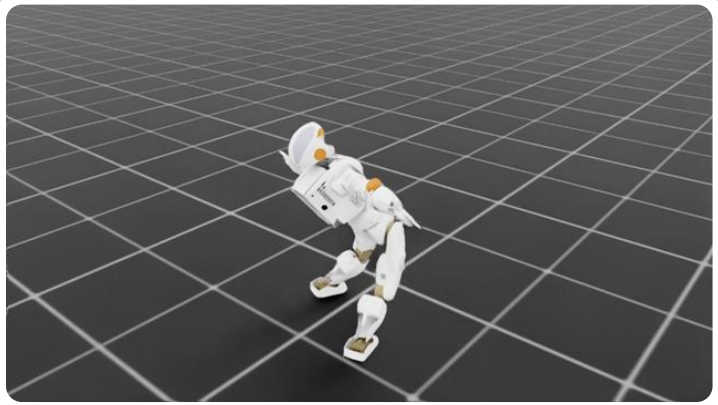
PPO

Stalls at 204-step attempts and drifts sideways almost as fast as it walks.



SAC

Walks 821 steps at 0.96 m/s. The only humanoid result worth showing.



TD3

Reaches 206 steps, then the run breaks down numerically.

Conclusion

Academic value

A repeatable way to teach reward design and compare training runs, without spending the class on setting tools up.

Commercial value

A faster way to get from a robot file to a working robot, before spending time on real hardware.

Project proof

A galloping Go1, a walking Go1, early humanoid walking, and a comparison of three algorithms that made SAC our default.

EasyRTG turns robot training from a pile of tools into a simple process you can follow.